

Evaluación de recuperación, golden sets y patrones avanzados

MMIA 6013 — IA Generativa y Agentes · Sesión 09 · jueves 24 de septiembre de 2026

Semana 2 · RA-3 · de los 110 minutos, 65 son actividad y 45 son este deck

Última sesión antes del Taller 2. El sábado es pasado mañana

Daniel Andrés Riofrío Almeida

Ayer cerré con «hoy no medimos nada». Hoy se mide

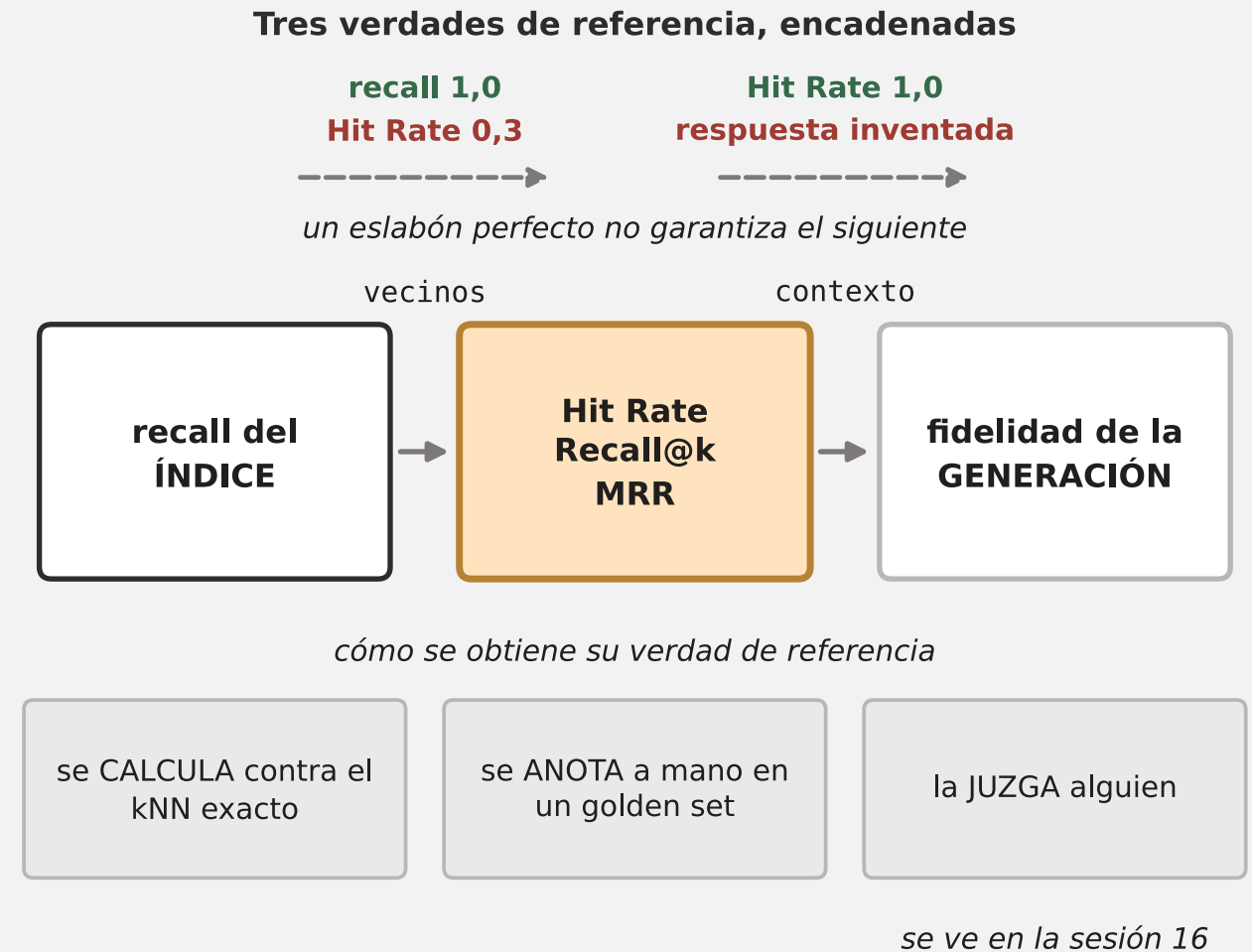
- Cierre literal de ayer: «Mañana: ¿este RAG es bueno? Hit Rate, MRR, Recall@k y golden sets»
- La regla de la semana: cualquier estrategia sofisticada debe ganarle con métricas. Hoy es el jueves
- Lo nuevo tiene tres caras. Una: la verdad de referencia se anota a mano, no se calcula
- Dos: esa anotación es un artefacto con dueño, versión y fecha de caducidad
- Tres: hay preguntas para las que no existe — y ahí terminan las métricas de hoy
- Nada del índice ni del chunking se reenseña: hoy solo se elige con evidencia medida

La verdad de referencia se anota

09

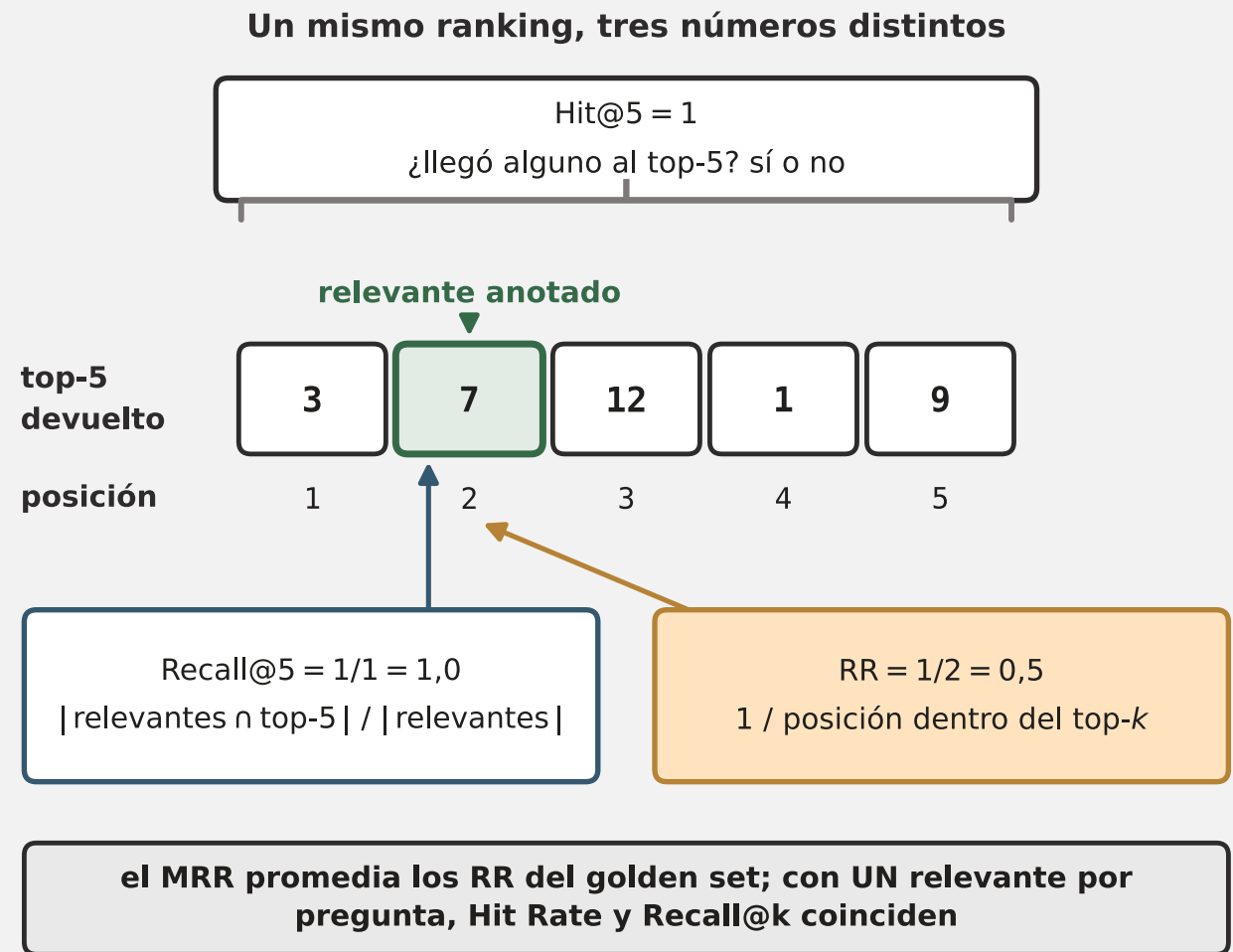
Un recall se calcula; el otro se anota

- Recall del índice (sesión 07): se compara contra el kNN exacto y se CALCULA
- Hit Rate, Recall@k y MRR (hoy): se comparan contra lo que un humano marcó y se ANOTAN
- Fidelidad de la generación: no se calcula ni se anota — alguien la JUZGA (final de hoy y sesión 16)
- Un índice con recall 1,0 puede alimentar un Hit Rate de 0,3, y un Hit Rate de 1,0 una respuesta inventada
- Lo que se calcula nunca se equivoca sobre lo que mide; lo que se anota puede quedar obsoleto
- Sin golden set, «mi RAG funciona bien» significa «probé tres preguntas y me gustó»



El mismo ranking, tres números distintos

- Relevante anotado: el fragmento 7. Ranking devuelto: 3, 7, 12, 1, 9. Con $k = 5$
- Hit Rate@5 = 1 — ¿llegó alguno al top- k ? Sí o no, y nada más
- Recall@5 = $|\text{relevantes} \cap \text{top-}k| / |\text{relevantes}| = 1/1 = 1,0$
- RR es *reciprocal rank*, rango recíproco: $1/\text{posición del primer acierto dentro del top-}k$, y 0 si no hay ninguno: $1/2 = 0,5$. El MRR, *mean reciprocal rank*, es su promedio
- Con UN relevante por pregunta, Recall@ k y Hit Rate son el mismo número
- La posición importa: el contexto se lee con sesgo de orden y cada fragmento extra cuesta tokens



MRR es el promedio de 1/posición, no la posición promedio

- Dos preguntas: el primer acierto cae en la posición 1 y en la posición 5
- Posición promedio: $(1 + 5)/2 = 3$
- MRR: $(1/1 + 1/5)/2 = (1 + 0,2)/2 = 0,6$ — que corresponde a la posición 1,67
- No son la misma cuenta y no ordenan igual: el recíproco premia siempre acertar arriba
- MAP, *mean average precision*: por pregunta se promedia la precisión en cada posición donde cayó un relevante (AP), y MAP es la media de las AP. Con UN relevante por pregunta, MAP = MRR
- nDCG, *normalized discounted cumulative gain*: para relevancia de grados. Suma lo que vale cada documento, descontado por su posición, y lo divide por lo que sumaría el ranking ideal

MRR = promedio de 1/posición

dos preguntas: el primer acierto cae en la posición 1 y en la 5

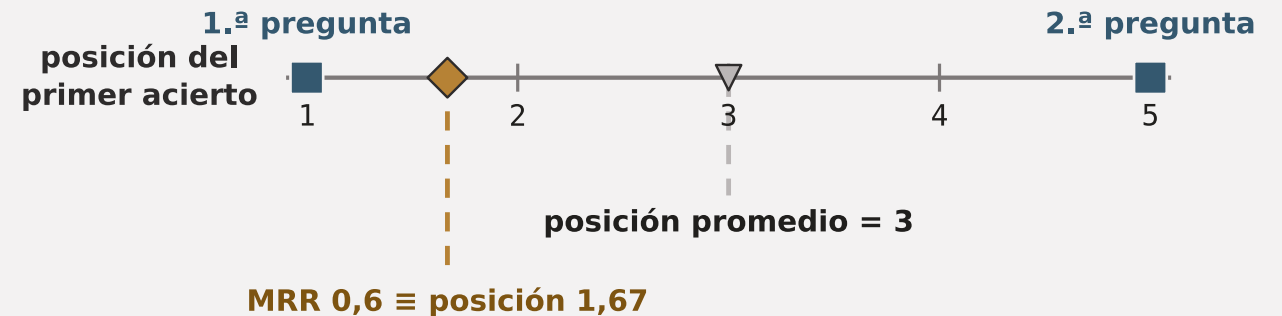
posición promedio

$$(1 + 5)/2 = 3$$

MRR

$$(1/1 + 1/5)/2 \\ = (1 + 0,2)/2 = 0,6$$

las dos cuentas parten de los mismos dos aciertos



Fuente: [curso/actividades/semana-2/s2-jue-explicacion.html](#), línea 52; [curso/actividades/_fuentes/s2-jue.py](#), `mrr()`; [curso/talleres/taller-02-rag-vector-search.md](#), §«Parte 2.b – Las métricas, sobre lo que cada una puede medir»; [fuentes/notas-teoricas/s2-rag-y-vector-search.md](#), §6.1, línea 212; [proceso/contexto/glosario.md](#), filas «MRR», «Average precision (AP) / MAP» y «nDCG». La aritmética del ejemplo es [AGREGADO], verificable a mano

Tres aserciones que se verifican a mano en cinco segundos

- Hit Rate es un booleano promediado: no le importa cuántos ni en qué posición
- Recall@k divide por el número de relevantes que ustedes anotaron — y una negativa anota cero
- Por eso las tres se calculan SOLO sobre las respondibles: el filtro por `if rel` es la lección, no una comodidad
- Sin ese filtro, la pregunta negativa que la actividad les exige revienta el notebook

```
# relevante en la posicion 2 -> RR = 1/2
assert mrr(["a", "b", "c"], ["b"], k=3) == 0.5
# fuera del corte no suma, ni al MRR ni al Hit Rate
assert mrr(["a", "b", "c"], ["b"], k=1) == 0.0
assert hit_rate_k(["a", "b", "c"], ["c"], k=2) == 0.0

def recall_k(rankings, relevantes, k=3):
    pares = [(r, rel) for r, rel in zip(rankings, relevantes) if rel]
    return float(np.mean(
        [len(set(r[:k]) & set(rel)) / len(rel) for r, rel in pares]))

# la negativa no cuenta. Sin el filtro: len(rel)==0, ZeroDivisionError
assert recall_k(["a"], ["b"], ["a"], [], k=1) == 1.0
```

Fuente: `curso/actividades/_fuentes/s2-jue.py`, ejercicio 1.1 —`hit_rate_k()`, `recall_k()`, `mrr()` y sus aserciones, en la revisión del 2026-09-20, la que corta el MRR en `k`—, recortado a 13 líneas, saltos reajustados a 72 columnas y acentos retirados, sin cambiar identificadores; `curso/planificacion/plan.md`, «Cerradas», 9 (discrepancia 0.v, CERRADA). La segunda frase del comentario final es [AGREGADO]: describe qué pasa sin el filtro

¿Está roto el índice?

Tu índice tiene recall 0,98 medido contra el kNN exacto, como el martes

Tu Hit Rate@5 sobre el golden set es 0,4

A. Sí: 0,98 no basta, hay que subir ef_search y volver a medir

*B. No: el índice es probablemente lo único que está bien

C. Los dos números miden lo mismo, así que uno de los dos está mal calculado

El golden set es un artefacto con dueño, versión y caducidad

09

Cuatro tipos de pregunta, y una que ustedes tienen que decidir

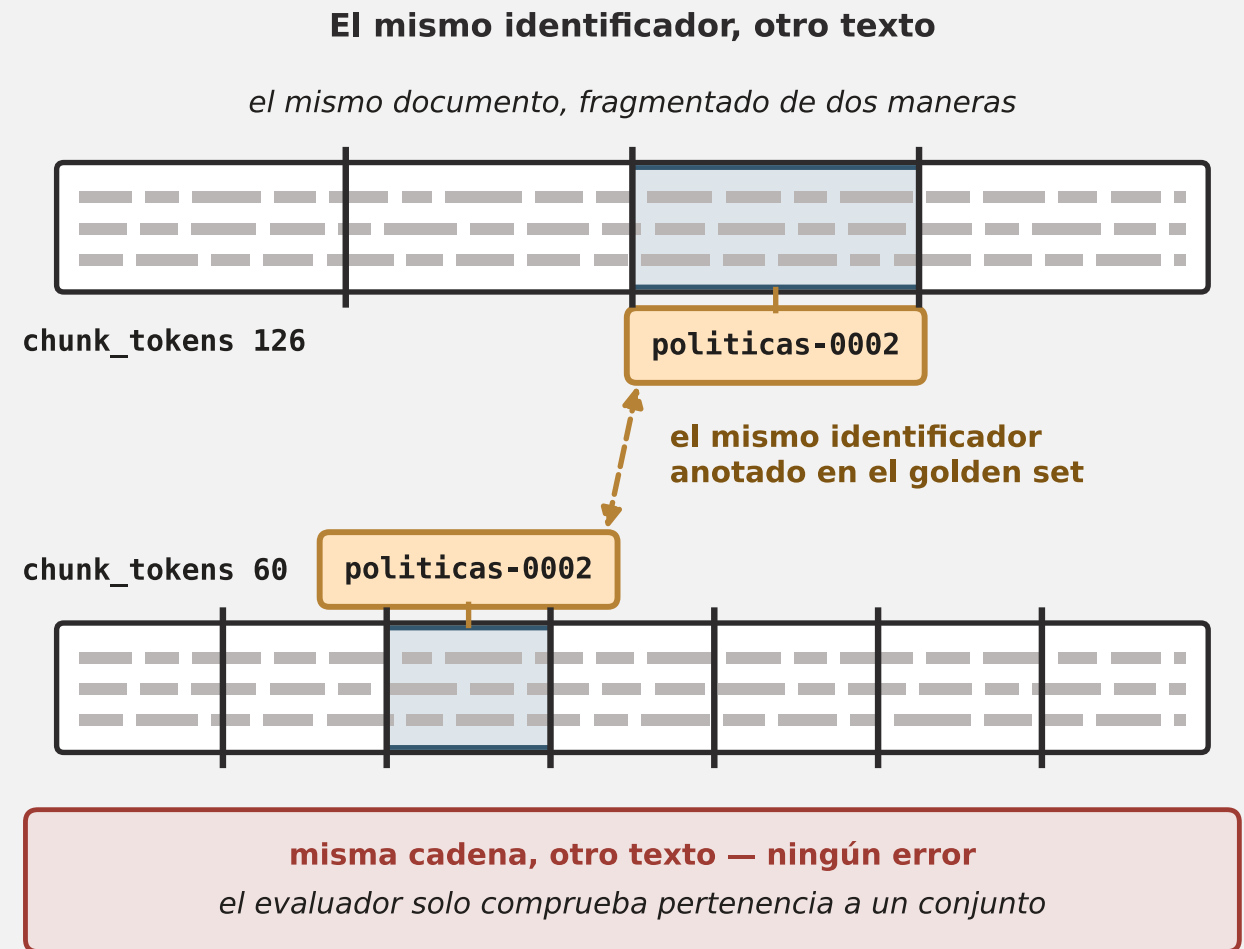
- simple — que encuentra lo evidente. Hit Rate y Recall@k coinciden
- multi-chunk — que junta piezas. Es el único tipo donde Recall@k $\neq$ Hit Rate
- negativa — que sabe no responder. El enunciado exige al menos 2 de 10
- adversarial — que no obedece una instrucción metida en la pregunta. NO es una negativa
- Y la plantilla les hace decidir: con documento fuente es responsable y cuenta en Hit Rate y MRR; sin él, cuenta como negativa. Decidan y díganlo en el informe
- ≥ 3 preguntas con vocabulario distinto al del documento: diez con las palabras exactas dan Hit Rate 1,0 y cero información

Manos al golden set: 10 preguntas para el corpus de ayer

- Evaluar recuperación pide tres piezas: corpus, preguntas y JUICIOS de relevancia. El corpus es el de ayer; las otras dos las hacen ustedes, ahora, en parejas
- 10 pares pregunta → documento(s) que la responden, con la respuesta esperada
- ≥ 3 con vocabulario distinto al del documento
- ≥ 2 sin respuesta en el corpus, para poder medir el «no sé»
- ≥ 1 cuya respuesta requiera dos documentos
- Anoten el DOCUMENTO y, si pueden, una FRASE literal suya: es lo que pide la plantilla del sábado. La razón viene en la próxima diapositiva

El mismo identificador, otro texto: el golden set caduca en silencio

- El chunker inventa el identificador, y su número depende de `chunk_tokens` y del solapamiento
- Anotar por ese identificador es comprobar pertenencia a un conjunto de cadenas: no puede notar el cambio
- Devuelve un número, no lanza ninguna excepción y el número no significa nada
- Añadir un documento NO invalida la anotación: el contador se reinicia en cada documento
- Cambiar la fragmentación SÍ la invalida, y en silencio. Es la Opción A del sábado
- Borrar o renombrar un documento anotado falla de forma visible: el identificador desaparece



Las dos puntas del cable, ya unidas: documento y frase literal

- En la versión anterior del Lab 02, el acierto era pertenencia a un conjunto de `chunk_id`, y la Opción A lo invalidaba
- Hoy hacen falta dos cosas: que el fragmento venga de un documento fuente y que contenga la frase anotada
- La frase se compara NORMALIZADA, así que sobrevive a tildes, a mayúsculas y a re-fragmentar
- Precio: con la frase vacía, cualquier fragmento del documento cuenta. Más robusto y menos informativo
- Elijan a sabiendas y déjenlo escrito: la frase tiene que caber ENTERA en un fragmento

```
# evaluation.py, version anterior: pertenencia a un conjunto de ids
expected = set(item.get("expected_chunk_ids", []))
retrieved = [h["chunk_id"] for h in hits]

# hoy: documento fuente + frase literal, comparada normalizada
def acierta(hit, item):
    if hit["document"] not in item.get("documentos_fuente", []):
        return False
    esperado = (item.get("fragmento_esperado") or "").strip()
    return not esperado or _normalizar_texto(esperado) in \
        _normalizar_texto(hit["text"])
```

Fuente: `curso/labs/Lab-02-RAG-VectorSearch/evaluation.py`, `acierta()` (versión actual; el bloque de arriba es la versión anterior, conservada para el contraste); `golden_set_plantilla.json`, campos `documentos_fuente` y `fragmento_esperado`; `rag_pipeline.py`, `retrieve()` y `_normalizar_texto()`; `curso/talleres/taller-02-rag-vector-search.md`, §«Parte 2.a» y §«Parte 3», Opción A; `curso/planificacion/plan.md`, «Cerradas», 10 (discrepancia 0.r, CERRADA)

El Hit Rate cayó de 0,8 a 0,3 al cambiar el chunking. ¿Lo publicas?

Tu golden set está anotado por `chunk_id`, como lo estaba la versión anterior del Lab 02
Cambiaste a fragmentación semántica para la Opción A y volviste a correr el evaluador
Escribiste en el informe: «el chunking semántico empeora la recuperación»

A. Publícalo: 0,3 contra 0,8 es una caída grande y el método es el mismo

B. Antes, comprueba cuántos de tus `expected_chunk_ids` siguen existiendo en el índice

C. Antes, sube `k` hasta que el número vuelva a parecerse al anterior

Las preguntas negativas y el número que no significaba lo que parecía

09

Aciertas TODAS las responsables y el script reporta 0,70. ¿Qué mejoras?

Corres el evaluador del Lab 02 en su versión anterior

Diez ítems: 4 simples, 3 multi-chunk, 2 negativas y 1 adversarial que anotaste sin documento fuente

Tu sistema acierta TODAS las que tienen respuesta en el corpus: ninguna falla es de recuperación

A. Sube k de 5 a 10: más candidatos, más aciertos

B. Cambia el modelo de embeddings por uno de más dimensiones

C. Nada del sistema: lo que hay que arreglar está en cómo se cuenta



El sistema que hacía lo correcto era penalizado dos veces

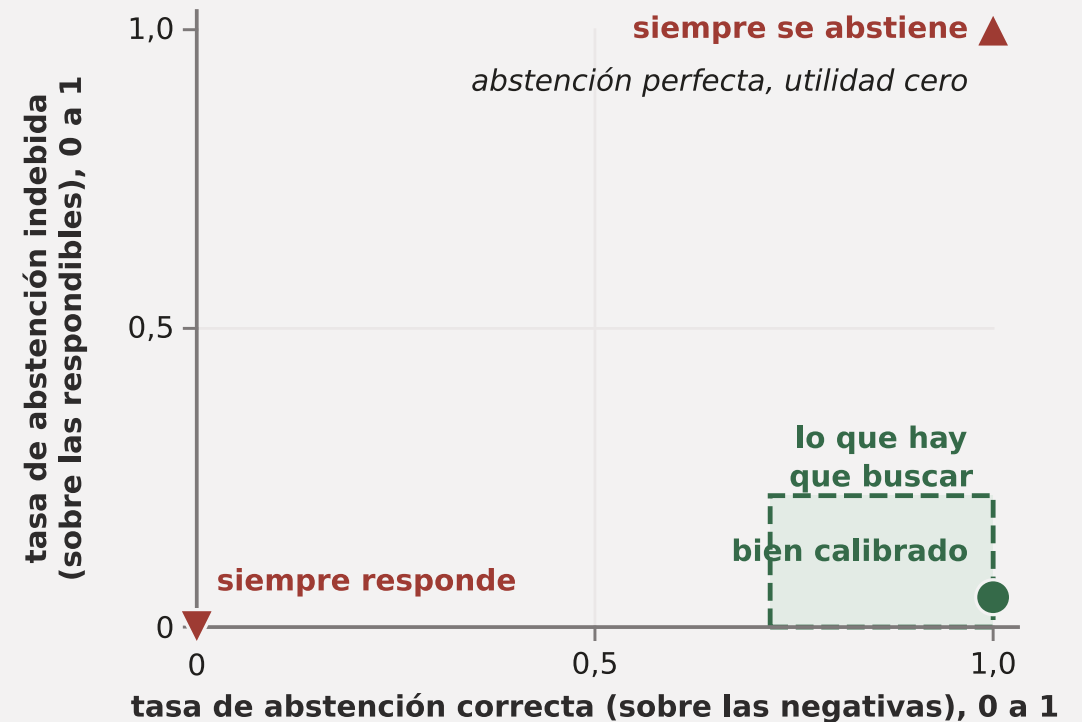
- El índice nunca dice «no sé»: retrieve no tiene umbral y devuelve k fragmentos igual
- La abstención es posterior: la ordena el prompt, o un umbral de distancia — que es una heurística, no una solución: en un corpus sin la respuesta los tres primeros vuelven a 1,298, 1,324 y 1,328
- En la versión anterior, una pregunta sin fragmento anotado daba acierto falso y rango recíproco 0,0
- Y se quedaba en el denominador del Hit Rate y del MRR: el que se abstenía bien perdía nota
- Hoy es_respondible() la saca de ese denominador, y answer() genera: ya hay conducta que juzgar
- La abstención pasó de no aparecer en el evaluador a ser dos tasas con su columna en resultados.csv

Fuente: curso/labs/Lab-02-RAG-VectorSearch/evaluation.py, es_respondible(), posicion_del_primer_acierto(), evaluate_retrieval() y escribir_csv() (versión actual; la conducta anterior está descrita en la cabecera del archivo, punto 1); rag_pipeline.py, retrieve(), build_prompt() y answer(); curso/talleres/taller-02-rag-vector-search.md, §«Parte 2.b – Las métricas, sobre lo que cada una puede medir»; Alammar & Grootendorst (2024), cap. 8, §«Caveats of dense retrieval», p. 234; curso/planificacion/plan.md, «Cerradas», 10 (discrepancias 0.n y 0.ab, CERRADAS)

Hacen falta dos tasas de abstención, no una — y ya están medidas

- Sobre las negativas: abstenciones correctas / número de negativas
- Sobre las respondibles: abstenciones indebidas / número de respondibles
- Hit Rate, Recall@k y MRR se calculan SOLO sobre las respondibles
- Un sistema que se abstiene siempre tiene la primera tasa en 1,0 y una utilidad de cero
- Es la simétrica de «un RAG que siempre responde algo es un generador de alucinaciones con citas»
- Las tres cosas ya están en evaluation.py, y el enunciado pide las dos tasas por su nombre

Una sola tasa no distingue calibrado de mudo



Hit Rate, Recall@k y MRR se calculan SOLO sobre las respondibles

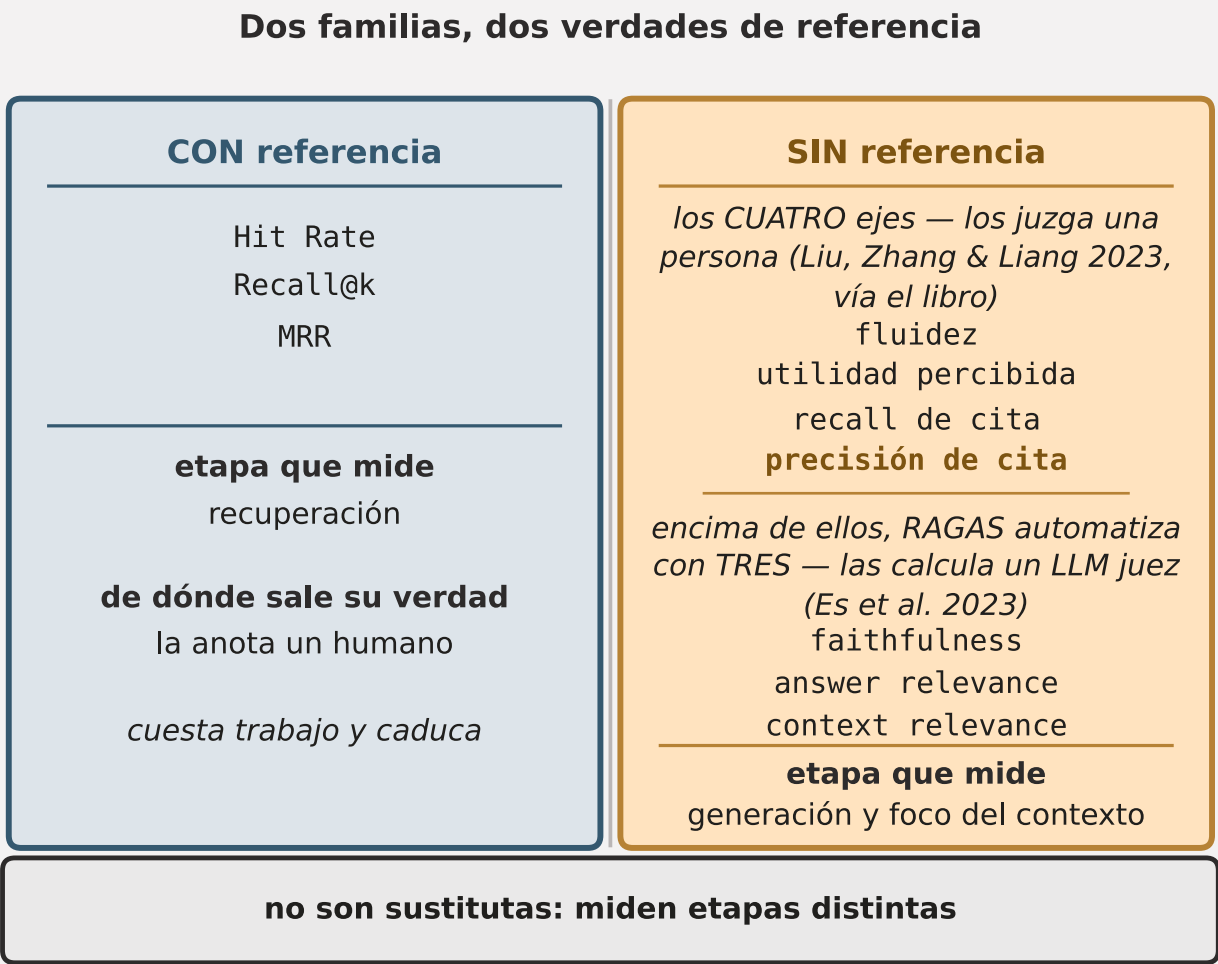
Fuente: curso/talleres/taller-02-rag-vector-search.md, §«Parte 2.b — Las métricas, sobre lo que cada una puede medir» (las dos tasas, literales); curso/labs/Lab-02-RAG-VectorSearch/evaluation.py, es_respondible(), evaluate_retrieval() y escribir_csv(); rag_pipeline.py, ABSTENCION y se_abstuvo(); curso/actividades/_fuentes/s2-jue.py, tasa_abstencion(); s2-mie.py, líneas 218–220, citado a través de slides_v2/sesion-08/contenido-base.md, §12.2

Dónde las métricas de hoy dejan de aplicar

09

Evaluar sin verdad de referencia

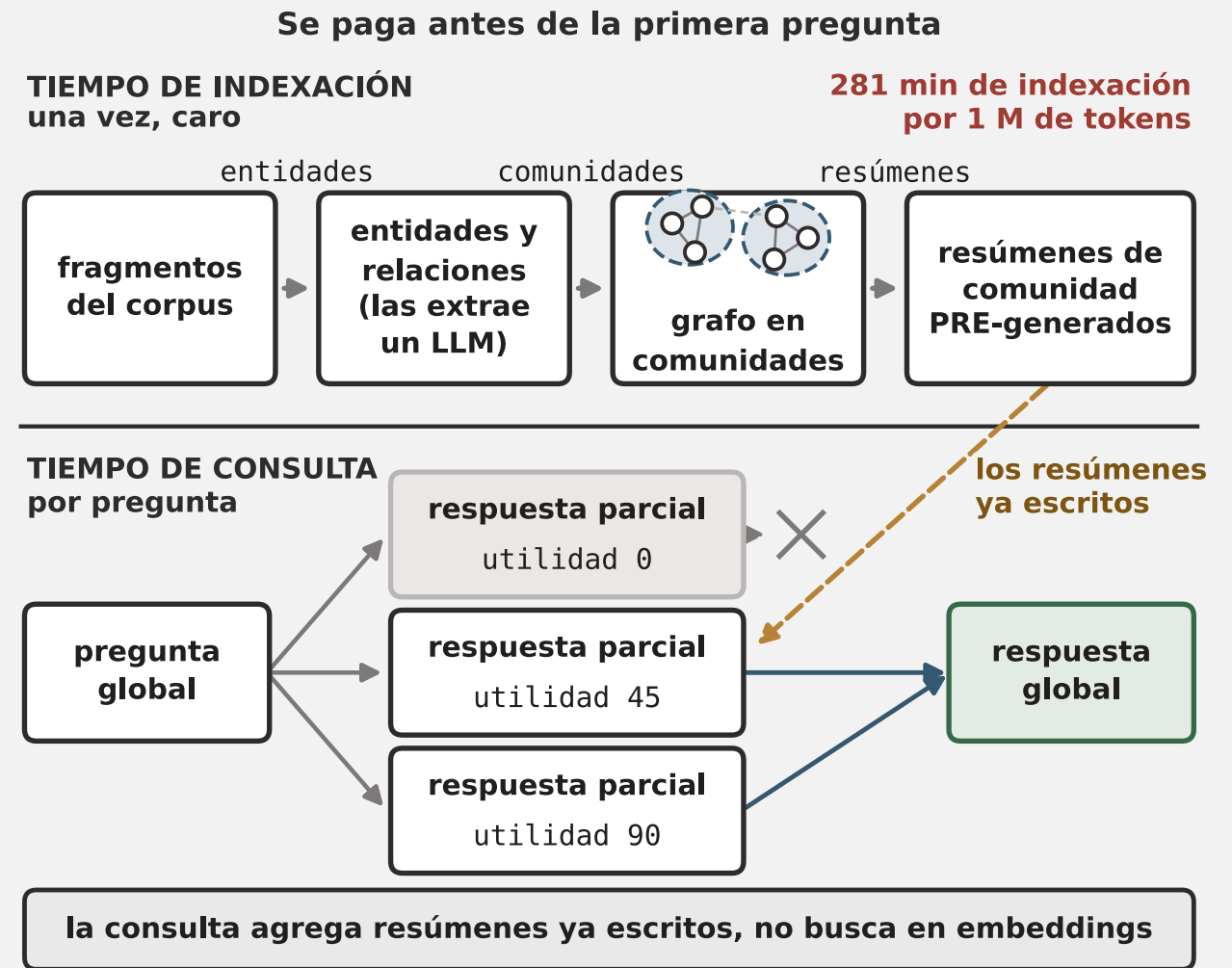
- Evaluar la respuesta pide CUATRO ejes —fluidez, utilidad percibida, recall de cita, precisión de cita— y los cuatro los juzga una persona
- Encima de ellos, RAGAS —*Retrieval Augmented Generation Assessment*, evaluación de RAG— automatiza TRES. Faithfulness = afirmaciones de la respuesta que se infieren del contexto / afirmaciones totales
- Answer Relevance: el LLM genera n preguntas desde la respuesta y se promedia el coseno con la original
- Context Relevance = oraciones cruciales / oraciones del contexto: mide FOCO, no cobertura — NO es Recall@k
- Precisión de cita: qué proporción de las citas que la respuesta pone sostienen de verdad lo que afirma. Es el número que faltaba
- Las tres de RAGAS las calcula un LLM juez, con sesgos de posición, verbosidad y autopreferencia: semana 4



Fuente: Es, James, Espinosa-Anke & Schockaert (2023), «Ragas», arXiv v2 (28 abr. 2025): abstract p. 1; §3, p. 3 y ec. 1; ec. 2, p. 4; §5, p. 5; Alammar & Grootendorst (2024), cap. 8, §«RAG Evaluation», p. 257, que atribuye los cuatro ejes a Liu, Zhang & Liang (2023), sin copia local y citado solo a través del libro; proceso/contexto/glosario.md, filas «RAGAS», «Context relevance» y «Citation precision». La librería del mismo nombre entra en la sesión 16: docs.ragas.io está registrada en fuentes/web/fuentes-web.md y NO fue consultada, así que aquí no se proyecta ninguna capacidad suya

GraphRAG: cuando no hay ningún fragmento que anotar

- «RAG fails on global questions... inherently a query-focused summarization task, rather than an explicit retrieval task»
- Para «¿cuáles son los temas principales del corpus?» no hay ningún documento fuente ni frase que anotar
- Hit Rate, Recall@k y MRR no están mal aquí: no están definidos
- Indexa caro: extrae entidades y relaciones, detecta comunidades y PRE-GENERA sus resúmenes
- Consulta: map-reduce sobre esos resúmenes, cada respuesta parcial con una utilidad de 0 a 100
- Gana en cobertura y diversidad, pierde en concisión, y cuesta 281 min de indexación por 1 M de tokens



Dentro del grafo, y cuatro formas de meterlo en un RAG

- Un LLM lee cada fragmento y extrae entidades, relaciones y afirmaciones (*claims*); los ejemplos few-shot del dominio deciden qué cuenta como entidad
- Las menciones repetidas se funden en un nodo por coincidencia exacta del nombre, y las veces que aparece una relación son el peso de su arista
- Leiden recursivo: cada nivel parte el grafo entero en comunidades que no se solapan, de C0 (raíz, pocas) a C3 (muchas y pequeñas)
- El nivel es un dial de costo: con C0, la consulta gasta más de un 97 % menos de tokens que resumir los textos fuente, y aún le gana al RAG vectorial en exhaustividad (72 %)
- La consulta global no es la única: también se entra por una entidad y se recorren sus vecinos, o se usa un grafo que ya existe como fuente o como índice
- Se elige por la pregunta: global, de un dato o que encadena entidades. Las dos últimas se pueden medir con el golden set de hoy; la global, no

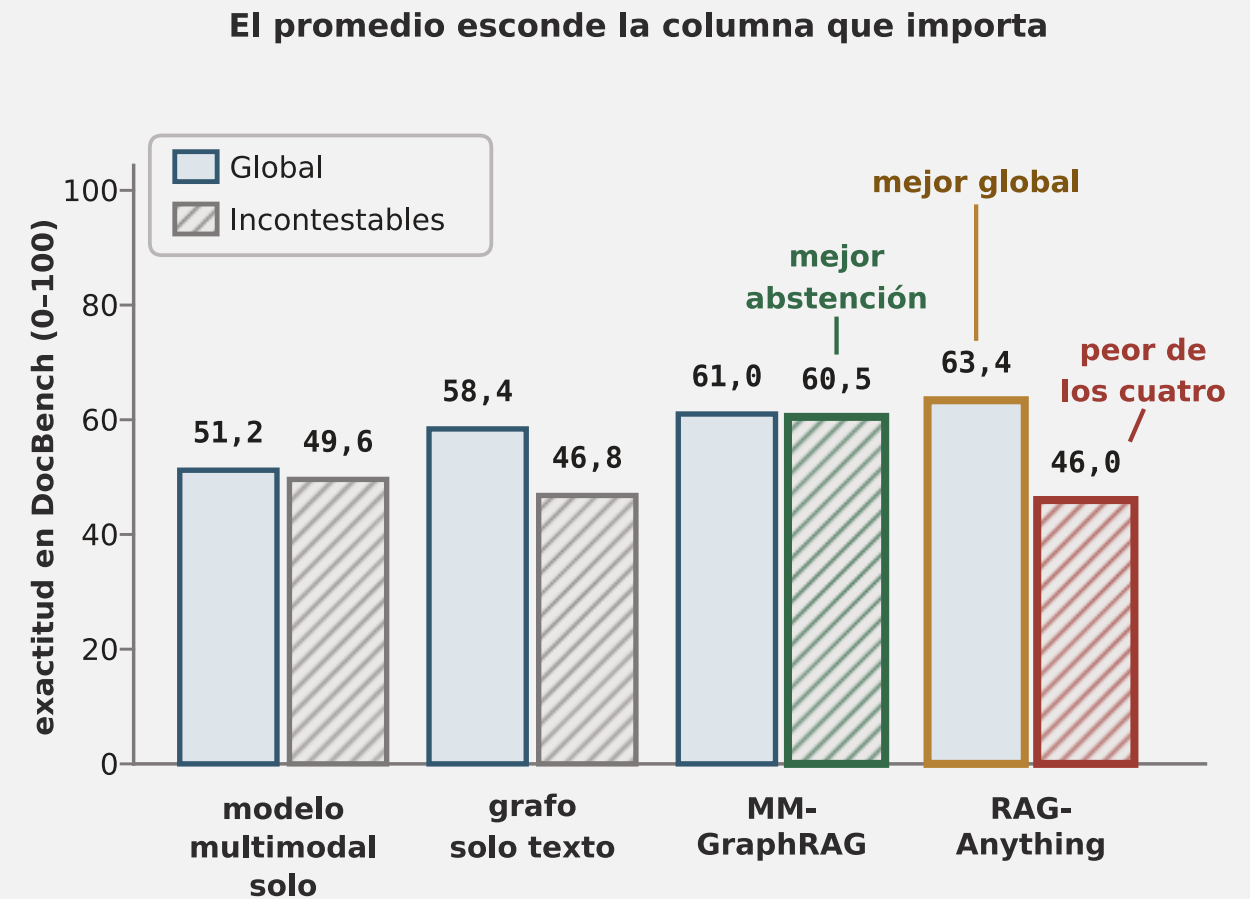
Cuatro formas de meter un grafo en un RAG

GLOBAL comunidades + map-reduce	recupera: nada; agrega resúmenes de comunidad ya escritos sirve para: «¿de qué trata el corpus?» <i>Edge et al. (2024) · la de la lámina anterior</i>
LOCAL entidad + sus vecinos	recupera: la entidad de la pregunta y sus vecinos a pocos saltos, más similitud densa sirve para: preguntas que encadenan entidades <i>Guo et al. (2025); en Edge et al., trabajo futuro</i>
KG COMO FUENTE un grafo que ya existe	recupera: entidades, tripletas o subgrafos sirve para: hechos verificados y precisos <i>Gao et al. (2023), §III.A</i>
KG COMO ÍNDICE enlaces entre documentos	recupera: párrafos, páginas o tablas unidos por similitud o por estructura sirve para: razonar a través de varios documentos <i>Gao et al. (2023), §III.B (KGP)</i>
no compiten: se elige por el tipo de pregunta	

Fuente: Edge et al. (2024), «From Local to Global», arXiv v2 (19 feb. 2025): §3.1.2–3.1.4, pp. 4–6; §4.1.2, p. 8; §5.1 y Tabla 2, p. 10; §6.2, p. 12; Guo, Ren, Xu, Zhang & Huang (2025), «RAG-Anything», §2.3, p. 6; Gao et al. (2023), «Retrieval-Augmented Generation for Large Language Models: A Survey», §III.A, pp. 7–8, y §III.B.3, p. 8. Tarjetas redibujadas, no copiadas. La última viñeta es [AGREGADO]

El promedio esconde justo lo que el sábado hay que medir

- RAG multimodal: tablas, figuras y ecuaciones como entidades de conocimiento, no como tipos de dato
- DocBench, 2025: el método con mejor exactitud global (63,4) es el PEOR en preguntas incontestables (46,0)
- El segundo en global (61,0) es el mejor reconociendo lo que el documento no responde (60,5)
- Ninguna de las dos cosas se ve en la columna «Global»
- En el mismo estudio, quitar el reordenador cuesta 1,0 punto (63,4 → 62,4): sus autores lo llaman marginal
- Reportar segmentado por tipo no es un lujo: es lo único que hace visible ese intercambio — y su evaluador ya lo imprime

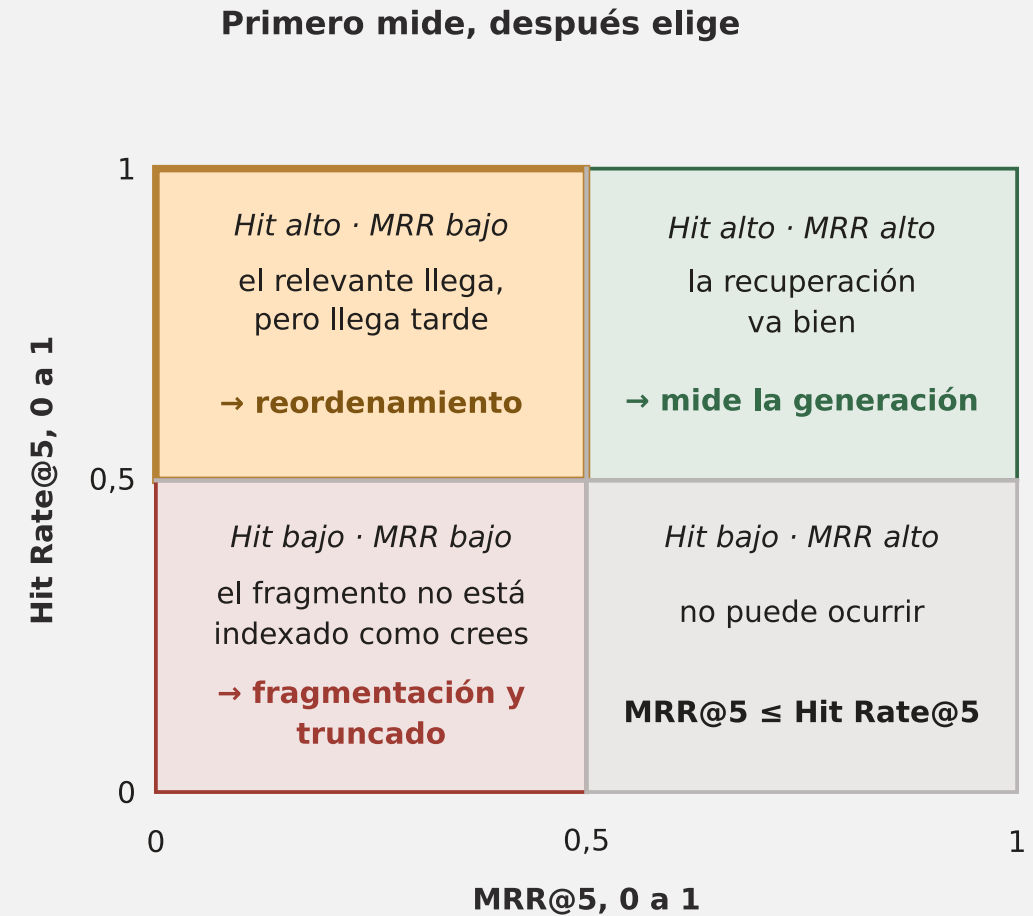


DocBench — Tabla 2 de Guo et al. (2025), redibujada

Fuente: Guo, Ren, Xu, Zhang & Huang (2025), «RAG-Anything», arXiv v1 (14 oct. 2025): abstract p. 1; §2.2–2.3, pp. 4–6; Tabla 2, p. 8 (cuatro filas citadas); §3.3 y Tabla 4, p. 9. Tablas redibujadas, no reproducidas. La lectura de la columna «Incontestables» es [AGREGADO]: el paper no la comenta. La segmentación por tipo la imprime `curso/labs/Lab-02-RAG-VectorSearch/evaluation.py, evaluate_retrieval()`, clave por_tipo

Primero mide, después elige. No al revés

- Hit Rate alto y MRR alto: la recuperación va bien. Lo que falta medir es la generación
- Hit Rate alto y MRR bajo: el relevante llega, pero llega tarde → reordenamiento
- Hit Rate bajo en siglas, códigos y números: es vocabulario, no semántica → híbrida con BM25 y RRF
- Hit Rate bajo en general: el fragmento no está indexado como crees → fragmentación y truncado
- Todo igual entre configuraciones: falta volumen. Con 10 preguntas y 2 negativas quedan 8 respondibles, y el Hit Rate solo puede moverse de $1/8 = 0,125$ en $0,125$
- El reordenamiento no sube el Hit Rate del conjunto de candidatos; sube el MRR — en MIRACL, 36,5 → 62,8 en nDCG@10 (libro del curso, 2024), que NO es la exactitud de DocBench



Fuente: [curso/actividades/semana-2/s2-jue-explicacion.html](#), líneas 104–108, 117 y 126; [curso/actividades/_fuentes/s2-jue.py](#), notas docentes y ejercicio 2.2 (la comparación de tres fragmentaciones); [slides_v2/sesion-08/contenido-base.md](#), §10.3; Alammari & Grootendorst (2024), cap. 8, §«Open source retrieval and reranking with sentence transformers», p. 243 (MIRACL, nDCG@10); [proceso/contexto/glosario.md](#), fila «nDCG». La tabla de diagnóstico y la resolución $1/n$ son [AGREGADO]

El sábado: un número que signifique algo

- Taller 2 — sistema RAG sobre base vectorial, 25 %, sábado 26 de septiembre
- Golden set de 10, anotado por documento y por frase literal; k=3 y k=5: llamen al evaluador dos veces
- Entreguen el resultados.csv crudo: de ahí salen todas las tablas, y sin él no hay reproducibilidad que calificar
- Elijan la extensión por el modo de falla que midieron, no por la técnica que les gusta
- Docker con Qdrant probado ANTES del sábado: no lo practicamos en ninguna sesión. RAG multimodal hoy se expone y no se evalúa
- La Parte 4 son dos preguntas: el 0,70 de hace un rato con SUS cifras, y una pregunta global sobre su corpus. Repasen GraphRAG esta noche